# 01 — TDD (Test-Driven Development)

Requisitos: **2** (Cadastrar cliente), **3** (Validar idade), **9** (Cadastrar conta corrente) e **12** (Transferir valores).

Ciclo aplicado: **RED → GREEN → REFACTOR**, um teste por vez.

Todas as saídas abaixo são trechos das execuções reais, salvas integralmente em `docs/evidencias/` (o número do arquivo indica a ordem cronológica). Comando usado em cada passo:

```
./mvnw test -Dtest=<ClasseDeTeste>      # RED e GREEN (só a classe do requisito)
./mvnw test                             # REFACTOR (suíte inteira)
java -cp target/classes teste.TesteUnitarioManual   # teste manual original, também no
REFACTOR
```

Testes criados (em `test/java/tdd/`):

| Requisito | Classe | Métodos |
|---|---|---|
| 2 | `CadastroClienteTest` | `deveCadastrarClienteValido`, `naoDeveCadastrarClienteComIdentificadorDuplicado` |
| 3 | `ValidacaoIdadeClienteTest` | `deveAceitarIdadeExatamenteNoLimiteInferior`, `deveAceitarIdadeExatamenteNoLimiteSuperior`, `naoDeveCadastrarClienteComIdadeAbaixoDoLimite`, `naoDeveCadastrarClienteComIdadeAcimaDoLimite`, `naoDevePermitirAlterarIdadeParaForaDaFaixa`, `devePermitirAlterarIdadeDentroDaFaixa` |
| 9 | `CadastroContaCorrenteTest` | `deveCadastrarContaCorrenteValida`, `naoDeveCadastrarNumeroDeContaDuplicado` |
| 12 | `TransferenciaTest` | `deveTransferirQuandoExisteSaldoSuficiente`, `naoDeveTransferirQuandoSaldoForInsuficiente`, `naoDeveTransferirValorNegativo`, `naoDeveTransferirValorZero` e, após a auditoria, `naoDeveTransferirParaDestinoNulo`, `naoDeveTransferirQuandoSaldoDoDestinoDeixariaDeSerFinito`, `naoDeveTransferirValorNaoNumerico` |
| 2 e 9 (auditoria) | `UnicidadeIdentificadorTest` | `naoDeveAdicionarDiretamenteNoRepositorioComIdDuplicado`, `naoDevePermitirAlterarAListaDoRepositorioPorFora`, `alterarAListaOriginalNaoDeveAfetarORepositorio`, `naoDeveCriarRepositorioComIdsDuplicados` |

---

## Requisito 2 — Cadastrar cliente

### Ciclo 2.1

#### Regra testada

Um cliente válido cadastrado no sistema deve poder ser consultado depois pelo seu id.

### RED

Teste criado: `deveCadastrarClienteValido()`

```
Cliente cliente = new Cliente(1, "Maria Silva", 31, "maria@gmail.com", 1, true);
boolean cadastrado = sistema.cadastrarCliente(cliente);
assertTrue(cadastrado);
assertSame(cliente, sistema.consultarCliente(1));
```

Resultado obtido — 1ª execução (`01-req02-red-compilacao.txt`):

```
[ERROR] .../test/java/tdd/CadastroClienteTest.java:[37,45] cannot find symbol
  symbol:   method cadastrarCliente(negocio.Cliente)
  location: variable sistema of type negocio.SistemaBancario
[INFO] BUILD FAILURE
```

Em seguida foi criado só o esboço do método, para o teste compilar (`return false;`, sem nenhuma lógica). 2ª execução (`02-req02-red-deveCadastrarClienteValido.txt`):

```
[ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0 <<< FAILURE! -- in
tdd.CadastroClienteTest
java.lang.AssertionError
[ERROR]   CadastroClienteTest.deveCadastrarClienteValido:40
```

Por que falhou: primeiro porque `SistemaBancario` não tinha operação de cadastro (erro de compilação). Depois, com o esboço, o `assertTrue(cadastrado)` da linha 40 falhou porque o método ainda não cadastrava nada.

### GREEN

Alteração realizada — o mínimo necessário:

```
public boolean cadastrarCliente(Cliente cliente) {
    gerClientes.adiciona(cliente);
    return true;
}
```

Resultado (`03-req02-green-...`):

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

### REFACTOR

Nenhuma refatoração relevante neste ciclo: o método tem duas linhas.

## Ciclo 2.2

### Regra testada

Não é permitido cadastrar um cliente com id já existente; o cliente original deve continuar lá.

### RED

Teste criado: `naoDeveCadastrarClienteComIdentificadorDuplicado()`

Resultado obtido (`04-req02-red-...`):

```
[ERROR] Tests run: 2, Failures: 1, Errors: 0, Skipped: 0 <<< FAILURE! -- in
tdd.CadastroClienteTest
java.lang.AssertionError
[ERROR]   CadastroClienteTest.naoDeveCadastrarClienteComIdentificadorDuplicado:58
```

Por que falhou: a implementação do GREEN anterior sempre adiciona e devolve `true`; o `assertFalse(cadastrado)` falhou.

### GREEN

Alteração realizada: verificar se o id já existe antes de adicionar.

```
if (gerClientes.pesquisa(cliente.getId()) != null)
    return false;
```

Resultado (`05-req02-green-...`):

```
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

### REFACTOR

Refatoração realizada: criado `boolean existe(int id)` em `Repositorio`/`RepositorioBase`, e o cadastro passou a usar `gerClientes.existe(cliente.getId())`. Também foi adicionado Javadoc ao método.

Justificativa: `pesquisa(id) != null` diz *como* verificar; `existe(id)` diz *o que* se quer saber. O método também será reaproveitado no cadastro de conta.

Resultado final (`06-req02-refactor-suite-completa.txt`):

```
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
TesteUnitarioManual -> Falharam: 0 / Passaram: 25
```

---

## Requisito 3 — Validar idade do cliente

### Regra existente (sem ciclo RED)

A análise mostrou que a regra **já existia**: o construtor de `Cliente` rejeita idades fora de **[18, 65]**. Antes de alterar qualquer coisa, foram escritos testes de **valor limite** (17, 18, 65, 66) para proteger essa regra:

`deveAceitarIdadeExatamenteNoLimiteInferior`, `deveAceitarIdadeExatamenteNoLimiteSuperior`, `naoDeveCadastrarClienteComIdadeAbaixoDoLimite`, `naoDeveCadastrarClienteComIdadeAcimaDoLimite`.

Resultado (`07-req03-testes-de-limite-regra-preexistente.txt`):

```
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
```

**Esses testes passaram na primeira execução.** Por isso **não** são apresentados como RED: eles caracterizam um comportamento já existente (testes de regressão). Seria desonesto apagar a regra só para "forçar" um RED.

## Ciclo 3.1

### Regra testada

A idade também precisa ser válida quando é **alterada**: `setIdade(17)` deve ser rejeitado e a idade anterior deve ser mantida.

### RED

Teste criado: `naoDevePermitirAlterarIdadeParaForaDaFaixa()`

```
Cliente cliente = criarClienteComIdade(30);
assertThrows(IdadeNaoPermitidaException.class, () -> cliente.setIdade(17));
assertEquals(30, cliente.getIdade());
```

Resultado obtido (`08-req03-red-...`):

```
[ERROR] Tests run: 5, Failures: 1, Errors: 0, Skipped: 0 <<< FAILURE! -- in
tdd.ValidacaoIdadeClienteTest
java.lang.AssertionError: expected negocio.IdadeNaoPermitidaException to be thrown, but
nothing was thrown
```

Por que falhou: `setIdade` apenas atribuía o valor (`this.idade = idade;`), sem aplicar a regra de idade. Um cliente válido podia ficar com idade inválida.

### GREEN

Alteração realizada: `setIdade` passou a declarar `throws IdadeNaoPermitidaException` e a repetir a verificação feita no construtor. Nenhum código de produção chamava `setIdade`, então a mudança de assinatura não quebrou nada.

Resultado (`09-req03-green-...`):

```
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

### REFACTOR

Refatoração realizada: a condição `idade < IDADE_MINIMA || idade > IDADE_MAXIMA` estava duplicada (construtor e setter). Foi extraída para `private static void validarIdade(int idade)`, usada nos dois lugares.

Justificativa: a regra de negócio "Validar idade do cliente" passa a ter **um único lugar** no código, com nome de domínio.

Resultado final (`10-req03-refactor-suite-completa.txt`):

```
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
TesteUnitarioManual -> Falharam: 0 / Passaram: 25
```

Depois do REFACTOR foi acrescentado um teste complementar, `devePermitirAlterarIdadeDentroDaFaixa` (setIdade(65)), para garantir que a validação não ficou restritiva demais. Ele passou de primeira, como esperado (aparece a partir de `15-...`).

---

## Requisito 9 — Cadastrar conta corrente

A conta não depende de Cliente no código atual (é o Cliente que guarda `idContaCorrente`), então essa dependência **não** foi introduzida.

### Ciclo 9.1

#### Regra testada

Uma conta válida cadastrada deve poder ser consultada depois pelo seu número (id).

#### RED

Teste criado: `deveCadastrarContaCorrenteValida()`. Como no requisito 2, o esboço `cadastrarConta(ContaCorrente)` com `return false;` foi criado junto com o teste para ele compilar.

Resultado obtido (`11-req09-red-...`):

```
[ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0 <<< FAILURE! -- in
tdd.CadastroContaCorrenteTest
java.lang.AssertionError
[ERROR]   CadastroContaCorrenteTest.deveCadastrarContaCorrenteValida:40
```

Por que falhou: o esboço não cadastrava e devolvia `false`.

#### GREEN

Alteração realizada: `gerContas.adiciona(conta); return true;`

Resultado (`12-req09-green-...`):

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

#### REFACTOR

Nenhuma refatoração relevante neste ciclo.

## Ciclo 9.2

### Regra testada

Não é permitido cadastrar duas contas com o mesmo número; a conta original e seu saldo devem permanecer.

### RED

Teste criado: `naoDeveCadastrarNumeroDeContaDuplicado()`

Resultado obtido (`13-req09-red-...`):

```
[ERROR] Tests run: 2, Failures: 1, Errors: 0, Skipped: 0 <<< FAILURE! -- in
tdd.CadastroContaCorrenteTest
java.lang.AssertionError
[ERROR]   CadastroContaCorrenteTest.naoDeveCadastrarNumeroDeContaDuplicado:58
```

Por que falhou: a conta duplicada era aceita (`true`).

### GREEN

Alteração realizada: `if (gerContas.existe(conta.getId())) return false;` (reaproveitando o `existe` criado no requisito 2).

Resultado (`14-req09-green-...`):

```
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

### REFACTOR

Refatoração realizada: `cadastrarCliente` e `cadastrarConta` ficaram com o **mesmo algoritmo** (verificar se existe → adicionar → devolver `true`). A lógica foi extraída para um único método privado:

```
private <T extends EntidadeBancaria> boolean cadastrar(Repositorio<T> repositorio, T entidade)
{
    if (repositorio.existe(entidade.getId()))
        return false;

    repositorio.adiciona(entidade);
    return true;
}
```

Justificativa: elimina a duplicação (DRY) aproveitando a interface `EntidadeBancaria` que o projeto já tinha.

> **Evolução posterior (auditoria):** a regra de unicidade foi levada para dentro do repositório (`Repositorio.adiciona` agora recusa id repetido; `id` virou `final`; a lista ficou protegida). Com

> isso, este método genérico ficou redundante e foi removido. Ver `06-correcoes-pos-auditoria.pdf` e a nova classe `UnicidadeIdentificadorTest`.

Resultado final (`15-req09-refactor-suite-completa.txt`):

```
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
TesteUnitarioManual -> Falharam: 0 / Passaram: 25
```

---

## Requisito 12 — Transferir valores entre contas

Situação inicial: a transferência existia em `GerenciadoraContas.transfereValor`, que alterava os saldos via `setSaldo`. Pela análise, a regra (saldo, débito, crédito) pertence à própria conta. Por isso a operação foi **conduzida por TDD na entidade**, como `ContaCorrente.transferir(destino, valor)`. No final, o repositório passou a delegar para ela.

### Ciclo 12.1

#### Regra testada

Com saldo suficiente, a transferência debita a origem e credita o destino.

#### RED

Teste criado: `deveTransferirQuandoExisteSaldoSuficiente()` (origem 100, destino 0, transferir 40)

1ª execução (`16-req12-red-compilacao.txt`):

```
[ERROR] .../TransferenciaTest.java:[25,44] cannot find symbol
[ERROR]   symbol:   method transferir(negocio.ContaCorrente,double)
[INFO] BUILD FAILURE
```

2ª execução, com o esboço `return false;` (`17-req12-red-...`):

```
[ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0 <<< FAILURE! -- in
tdd.TransferenciaTest
java.lang.AssertionError
[ERROR]   TransferenciaTest.deveTransferirQuandoExisteSaldoSuficiente:28
```

Por que falhou: a conta não tinha a operação e, depois, o esboço não movimentava saldo.

#### GREEN

Alteração realizada — o mínimo, **sem** verificar saldo ainda:

```
public boolean transferir(ContaCorrente destino, double valor) {
    this.saldo -= valor;
    destino.saldo += valor;
    return true;
}
```

Resultado (`18-req12-green-...`): `Tests run: 1, Failures: 0` — `BUILD SUCCESS`.

### REFACTOR

Nenhuma refatoração relevante (três linhas).

## Ciclo 12.2

### Regra testada

Sem saldo suficiente, a transferência é rejeitada e os saldos não mudam.

### RED

Teste criado: `naoDeveTransferirQuandoSaldoForInsuficiente()` (transferir 150 com saldo 100)

Resultado (`19-req12-red-...`):

```
[ERROR] Tests run: 2, Failures: 1, Errors: 0, Skipped: 0 <<< FAILURE! -- in
tdd.TransferenciaTest
java.lang.AssertionError
[ERROR]   TransferenciaTest.naoDeveTransferirQuandoSaldoForInsuficiente:45
```

Por que falhou: a implementação mínima do ciclo anterior transferia sempre (a origem ficaria com -50).

### GREEN

Alteração realizada: `if (this.saldo < valor) return false;` antes do débito.

Resultado (`20-req12-green-...`): `Tests run: 2, Failures: 0` — `BUILD SUCCESS`.

### REFACTOR

Refatoração realizada:

- produção: a regra ganhou nomes do domínio: `possuiSaldoSuficiente(valor)`, `debitar(valor)` e `creditar(valor)`;
- testes: as contas passaram a ser criadas em `@Before`, e a verificação "saldos inalterados" virou o método auxiliar `assertSaldosInalterados()`.

Justificativa: o código passa a usar a linguagem do negócio ("se possui saldo suficiente, debita a origem e credita o destino"), e os testes deixam de repetir o *Arrange*.

Resultado (`21-req12-refactor-ciclo2-suite-completa.txt`): `Tests run: 12, Failures: 0` — `BUILD SUCCESS` — manual 25/25.

## Ciclo 12.3

### Regra testada

Valor negativo não pode ser transferido.

### RED

Teste criado: `naoDeveTransferirValorNegativo()` (transferir -50)

Resultado (`22-req12-red-...`):

```
[ERROR] Tests run: 3, Failures: 1, Errors: 0, Skipped: 0 <<< FAILURE! -- in
tdd.TransferenciaTest
java.lang.AssertionError
[ERROR]   TransferenciaTest.naoDeveTransferirValorNegativo:58
```

Por que falhou: `100 >= -50` é verdadeiro, então a verificação de saldo deixava passar. A origem seria **creditada** e o destino **debitado**. O mesmo defeito existia no `transfereValor` original.

### GREEN

Alteração realizada — mínima: `if (valor < 0) return false;` (de propósito **sem** tratar o zero ainda).

Resultado (`23-req12-green-...`): `Tests run: 3, Failures: 0` — `BUILD SUCCESS`.

### REFACTOR

Nenhuma neste ciclo; a próxima regra (zero) mexe na mesma linha.

## Ciclo 12.4

### Regra testada

Valor zero não é uma transferência válida.

### RED

Teste criado: `naoDeveTransferirValorZero()`

Resultado (`24-req12-red-...`):

```
[ERROR] Tests run: 4, Failures: 1, Errors: 0, Skipped: 0 <<< FAILURE! -- in
tdd.TransferenciaTest
java.lang.AssertionError
[ERROR]   TransferenciaTest.naoDeveTransferirValorZero:69
```

Por que falhou: a guarda do ciclo anterior era `valor < 0`, então 0 passava e a operação devolvia `true`.

### GREEN

Alteração realizada: `valor < 0` → `valor <= 0`.

Resultado (`25-req12-green-...`): `Tests run: 4, Failures: 0` — `BUILD SUCCESS`.

### REFACTOR

Refatoração realizada:

1. As duas guardas foram unidas em uma linha legível, com o método `valorPositivo(valor)`:

```
public boolean transferir(ContaCorrente destino, double valor) {
    if (!valorPositivo(valor) || !possuiSaldoSuficiente(valor))
        return false;

    this.debitar(valor);
    destino.creditar(valor);
    return true;
}
```

2. `GerenciadoraContas.transfereValor` deixou de duplicar a regra: agora só localiza as contas e chama `contaOrigem.transferir(contaDestino, valor)`.

Justificativa: a regra fica em um único lugar (a entidade). Além disso, o defeito do valor negativo/zero foi corrigido também no caminho antigo, usado pelo `TesteUnitarioManual`.

Resultado final (`26-req12-refactor-final-suite-completa.txt`):

```
[INFO] Tests run: 2, ... -- in tdd.CadastroClienteTest
[INFO] Tests run: 6, ... -- in tdd.ValidacaoIdadeClienteTest
[INFO] Tests run: 4, ... -- in tdd.TransferenciaTest
[INFO] Tests run: 2, ... -- in tdd.CadastroContaCorrenteTest
[INFO] Tests run: 14, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
TesteUnitarioManual -> Falharam: 0 / Passaram: 25
```

---

### Ciclos adicionais (após auditoria)

Dois ciclos RED → GREEN foram feitos depois, a partir da auditoria: `naoDeveTransferirParaDestinoNulo` (a NPE ocorria **depois** do débito) e `naoDeveTransferirQuandoSaldoDoDestinoDeixariaDeSerFinito`. O REFACTOR reuniu todas as condições em `transferenciaPermitida(destino, valor)`, verificada antes de qualquer alteração de saldo. Também foi adicionado `naoDeveTransferirValorNaoNumerico` (passou de primeira). Detalhes e evidências 33 a 37 em `06-correcoes-pos-auditoria.pdf`.

---

## Resumo dos ciclos

| Requisito | Ciclos RED→GREEN | REFACTOR relevante |
|---|---|---|
| 2 | 2 (cadastro válido, id duplicado) | `Repositorio.existe(id)` |
| 3 | 1 (setIdade) + 4 testes de limite de regra pré-existente | `validarIdade()` único |
| 9 | 2 (cadastro válido, número duplicado) | método genérico `cadastrar()` (depois substituído, ver abaixo) |
| 12 | 4 (saldo suficiente, insuficiente, negativo, zero) | `possuiSaldoSuficiente`/`debitar`/`creditar`/`valorPositivo`; repositório delega para a entidade |
| 2 e 9 (auditoria) | 1 ciclo com 4 testes (`UnicidadeIdentificadorTest`) | unicidade garantida no repositório; `cadastrar()` genérico removido |

| Requisito | Ciclos RED→GREEN | REFACTOR relevante |
|---|---|---|
| 12 (auditoria) | 2 (destino nulo, saldo não finito) | `transferenciaPermitida()` |
| 9 e 12 (2ª auditoria) | 1 ciclo com 3 testes (`SaldoFinitoTest`) | nenhum necessário; invariante `validarSaldo()` |